Proje Raporu — AI Model Güvenlik Projesi

> **Ne için:** Projenin **başından bu yana** ne yapıldığını, her kararın **neden/nasıl**'ını öğrenme amacıyla toplar. Öğrendiklerinle kıyaslayıp eksikleri görmen için yazıldı. Tarih: 2026-10-10 · Durum: Bölüm 1 çekirdek tamam, Bölüm 2 iskelet hazır.

---

0. Yönetici özeti

Soru: İndirilen bir yapay zeka modeli, beraberinde virüs/backdoor getiriyor ya da arka planda zararlı bir servis çalıştırıyor olabilir mi? Bunu tespit eden bir araç yapmak.

Proje **iki bölüm**:

Bölüm	Ne	Durum
1 — modelguard	Model dosyalarını **statik + dinamik** analizle tarayan savunma aracı	✅ Çekirdek tamam, test edildi, araçlarla kıyaslandı
2 — Backdoor çalışması	Bir modele nesne-tetikleyicili **davranışsal** backdoor öğretip, bunun Bölüm 1'den neden **kaçtığını** göstermek	🟡 Kod iskeleti hazır; eğitim verisi (fotoğraf) + eğitim sana bağlı

Ana tez: modelguard, dosyadaki *kod* tehditlerini yakalar. Bölüm 2'nin ürettiği backdoor ise ağırlıklara gömülü bir *davranıştır* — dosyada zararlı kod yoktur. Bu yüzden statik tarama onu göremez. **Bu sınır bilinçli olarak gösterilecek.**

Sayılarla: **18 commit**, **~4100 satır** uygulama kodu (testlerle ~5900), **104 test**: 103'ü geçiyor, 1'i atlanıyor (Docker kapalıyken konteyner testi).

---

1. Neden "model indirmek" risklidir?

Bir model indirince aslında **iki şey** iner:

1. **Ağırlıklar** (öğrenilmiş sayılar) — zararsız görünür ama davranış saklayabilir (Bölüm 2).
2. **Yükleme mantığı** — modeli belleğe almak için **kod çalıştırılır**. İşte asıl risk burada.

En yaygın tuzak **pickle**'dır. Python'ın `pickle` formatı nesneleri "seri" hale getirir; ama geri yüklerken (`unpickle`) dosyanın içindeki talimatlara göre **fonksiyon çağırabilir**. Kötü niyetli bir `.pkl` / `.pt` dosyası, yüklenirken `os.system("…")` çağırıp komut çalıştırabilir (uzaktan kod yürütme = RCE). PyTorch `.pt`, scikit-learn `.pkl`, çoğu eski format pickle temellidir.

Not: PyTorch 2.6'dan beri `torch.load` varsayılan olarak `weights_only=True` ile çalışır ve keyfi global'leri reddeder. Bu riski azaltır ama ortadan kaldırmaz. Eski sürümler, `weights_only=False` diyen kodlar, joblib/scikit-learn gibi diğer pickle yükleyicileri hâlâ risk altındadır. Bu yüzden yüklemeden **önce** tarama değerlidir.

> **Öğrenme kutusu:** pickle opcode'ları (`GLOBAL` , `REDUCE` , `STACK_GLOBAL`), `__reduce__` metodu, "safetensors neden güvenli" (küçük bir JSON başlık + ham tensör baytları taşır; yüklerken çalıştırılacak kod yoktur).

---

2. Bölüm 1 — modelguard mimarisi

Araç, sorumlulukları ayrılmış **katmanlardan** oluşur. Bu ayrım, yeni format eklemeyi kolaylaştırır ve her parçayı tek tek test etmeyi sağlar.

```
modelguard/
  cli.py        komut satırı: scan <yol> [--json] [--html] [--dynamic] [--allow-host]
  models.py     ortak veri tipleri: Severity, Finding, FileReport, ScanResult
  formats.py    magic-byte ile GERÇEK format tespiti (uzantıya güvenmez)
  scanner.py    orkestratör: her dosyayı doğru analizörlere dağıtır
  scoring.py    bulgulardan düzey + MITRE ATLAS / OWASP çerçeve eşlemesi
  report.py     terminal (rich) + JSON + HTML rapor
  htmlreport.py jinja2 ile tek-dosya HTML (tarama + karşılaştırma)
  analyzers/    format-özel statik analizörler (aşağıda)
  dynamic/      izole çalıştırma: monitor + sandbox + sysdiff
```

Tasarım ilkeleri (projenin omurgası)

1. **Statik analiz modeli ASLA yüklemez/çalıştırmaz** — yalnızca baytları/opcode'ları okur.
2. **"Taranamadı" ≠ "güvenli"** — çözülemeyen/bozuk bir şey şüpheli sayılır, sessizce geçilmez.
3. **Denylist değil allowlist** — "şu kötüler" listesi yerine "yalnızca şunlar güvenli" yaklaşımı.
4. **Tarayıcı, gerçek yükleyicinin okuyabildiğini okuyabilmeli** — aksi halde atlatılır.
5. **Model kaynaklı içerik rapora escape edilerek yazılır** — rapora enjeksiyon olmasın.

2.1 Format tespiti (`formats.py`) — neden magic byte?

Uzantı yalan söyler: kötü bir pickle `.bin` , `.pt` , hatta `.safetensors` adıyla gelebilir. Bu yüzden dosyanın **ilk baytlarına** (magic signature) bakılır:

İlk baytlar	Format
`PK\x03\x04`	ZIP (çoğu PyTorch `.pt`)
`7z\xbc\xaf…`	7z arşiv
`\x89HDF…`	HDF5 (Keras `.h5`)
`GGUF`	llama.cpp GGUF
`\x80`	pickle (PROTO opcode)
`\x7fELF` / `MZ`	**çalıştırılabilir** — bir modelde ASLA olmamalı → doğrudan MALICIOUS

2.2 Analizörler (her format için bir uzman)

Analizör	Hedef	Ne arar
pickle (çekirdek)	`.pkl .pt .bin`, ZIP/7z/TAR üyeleri	opcode akışında tehlikeli global (`os.system`, `subprocess…`) + çağırma kalıbı; allowlist dışı global
keras	`.keras .h5`	`Lambda` katmanı (keyfi kod), güvensiz deserialization, gerçek custom kayıtlı nesneler
gguf	`.gguf`	metadata'daki `chat_template` içinde Jinja SSTI (şablon enjeksiyonu) kalıpları
savedmodel	TF `saved_model.pb`	graf op listesinde tehlikeli op'lar (ör. dosya okuma/yazma, `PyFunc`)
remotecode	repo `.py` dosyaları	`trust_remote_code` modüllerinde AST ile tehlikeli çağrı (import takma adı çözümlü)

2.3 Skorlama ve çerçeve eşlemesi (`scoring.py`)

Her bulgu bir **Severity** taşır: `SAFE(0) < LOW(1) < SUSPICIOUS(2) < MALICIOUS(3)`. Dosyanın/taramanın düzeyi = en yüksek bulgu düzeyi. CLI **çıkış kodu = düzey** (CI/script için). Bulgular ayrıca **MITRE ATLAS** teknikleri ve **OWASP ML** riskleriyle etiketlenir — profesyonel güvenlik diliyle konuşmak için.

2.4 Rapor (`report.py` , `htmlreport.py`)

Üç çıktı: renkli **terminal** (rich), makine-okur **JSON**, paylaşılabilir **HTML** (`--html`). HTML, jinja2 ile üretilir ve **autoescape** sayesinde model kaynaklı metin (ör. kötü bir başlık/kanıt) HTML'e ham geçmez — script enjeksiyonu önlenir.

---

3. İnceleme turu — yanlış alarmlar ve kaçırmalar

İlk sürümden sonra analizörler gözden geçirildi. İki tür hata bulundu:

- **Yanlış alarm (yanlış-pozitif):** temiz bir dosya şüpheli çıkıyor.
- **Kaçırma (yanlış-negatif):** zararlı bir dosya temiz çıkıyor. Bir güvenlik aracı için daha tehlikeli olan budur.

Yanlış alarmların önemli bir kısmı analizörler **gerçek, temiz** dosyalarda denenince ortaya çıktı (Qwen2.5 ve DeepSeek-R1-Distill chat şablonları, Phi-3 modeling dosyası, Keras kaynak kodu). Sentetik testler bunları göstermiyordu.

Commit	Tür	Sorun	Çözüm
`443fd64`	Kaçırma + yanlış alarm	`from os import system` / `import subprocess as sp` gibi takma adlı çağrılar eşleşmiyor, zararlı dosya SAFE çıkıyordu; ayrıca gerçek Phi-3 dosyası SUSPICIOUS çıkıyordu	takma ad çözümleyici (`_ImportResolver`); `importlib` / `compile` / `getattr` önemleri düşürüldü
`fd962d7`	Yanlış alarm + kaçırma	ZIP olmayan eski torch formatı (5 ardışık pickle + ham tensör) her zaman SUSPICIOUS çıkıyordu; eski TAR formatındaki uzantısız `pickle` / `sys_info` üyeleri hiç taranmıyordu (CVE-2025-32434'ün kullandığı format)	sihirli sayı tanınır, PyTorch gibi tam 5 akış taranır; uzantısız TAR üyeleri taranır
`199d1f7`	Yanlış alarm	Keras 3 yerleşik nesnelere de (`relu`, `Functional`) `registered_name` yazar; sıradan modeller SUSPICIOUS çıkıyordu	yalnızca gerçek custom kayıt biçimi `paket>Ad` işaretlenir
`4040d3a`	Yanlış alarm	her TF SavedModel'de bulunan `SaveV2` / `RestoreV2` / `MergeV2Checkpoints` op'ları SUSPICIOUS sayılıyordu	checkpoint op'ları LOW; `ReadFile` / `WriteFile` / `PyFunc` MALICIOUS kalır
`9b3b035`	Yanlış alarm	gerçek DeepSeek-R1-Distill şablonu `namespace()` kullandığı için, araç çağrısı şablonları da `a.b.c` zincirleri yüzünden SUSPICIOUS çıkıyordu	`namespace()` meşru sayıldı; zincirler LOW; yalnızca `{% import %}` / `{% from … import %}` MALICIOUS; gerçek şablonlar test verisi yapıldı
`42e1e66`	Kaçırma	`.keras` adıyla gelen bir **PyTorch ZIP**'i yalnızca keras analizörüne gidiyor, içindeki `data.pkl` hiç taranmıyordu	`.keras` dosyaları pickle analizöründen de geçirilir

> **Alınan dersler:** (1) Yeni bir kuralı **mutlaka gerçek temiz örneklerle** de dene; yoksa araç her şeye alarm verir ve kimse ona güvenmez. (2) Kaçırmalar genellikle "araç dosyayı yükleyiciden farklı okuyor" durumundan doğar (takma adlar, uzantısız TAR üyeleri, yanlış uzantı). Bu yüzden 4 numaralı tasarım ilkesi var.

---

4. Dinamik analiz — "arkada servis çalışıyor mu?"

Statik analiz kodu *okur*; dinamik analiz modeli **izole bir ortamda gerçekten yükleyip** ne yaptığını *izler*. Bu, kullanıcının asıl sorusunu doğrudan yanıtlar.

```
dynamic/
  monitor.py   sandbox İÇİNDE çalışır: audit hook ile os.system/socket/subprocess vb.
               olayları yakalar. Güvenlik kapısı: MODELGUARD_SANDBOX=1 yoksa çalışmaz.
  sandbox.py   orkestratör: Docker backend (ağ kapalı, salt-okunur, limitli) VEYA
               --allow-host (zayıf). Süreç ağacı temizliği.
  sysdiff.py   yükleme ÖNCESİ/SONRASI süreç + dinlenen port farkı (psutil).
```

4.1 İki backend

- **Docker (tercih edilen):** `--network none` (ağ yok), `--read-only`, bellek/pid limiti, zaman aşımı. Gerçek izolasyon. Gerçek indirilmiş modeller **yalnızca** bununla denenmeli.
- **Host (`--allow-host`, zayıf):** modeli bu makinede çalıştırır. Yalnızca test/geliştirme.

4.2 Süreç ağacı temizliği (`8842e0b`)

Sorun: host modunda payload'ın başlattığı arka plan servisi, tarama bitince de yaşamaya devam ediyordu — tam da yakalamak istediğimiz senaryo! **Kök neden:** payload monitörün stdout borusunu miras alıyor, `communicate()` torunun ölümünü bekliyordu. **Çözüm:** sonuç artık bir **dosyaya** yazılır; Windows'ta alt süreç bir **Job Object**'e atanır (çıkışta tüm ağaç, ayrılmış torunlar dahil, sonlandırılır); POSIX'te `killpg`.

4.3 Sistem farkı — sysdiff (`f90281d`)

Model yüklenmeden önce ve sonra sistemin **fotoğrafı** alınır (çalışan süreçler + dinlenen portlar). Yüklemeden sonra ortaya çıkıp **hâlâ çalışan**: - **yeni dinlenen port** → MALICIOUS (olası arka kapı/servis), - **yeni süreç** → SUSPICIOUS (olası kalıcılık). Bu bulgu, aynı anda başlayan ilgisiz sistem süreçlerini de içerebilir; rapor bunu açıkça belirtir.

sysdiff **yalnızca host backend'inde** çalışır. Docker'da konteyner kapanınca içinde başlayan her şey zaten ölür. Bu nedenle orada "arkada kalan süreç" sorusu, konteyner içindeki audit hook olaylarıyla (ör. `socket.bind`, `subprocess.Popen`) yanıtlanır.

4.4 Dinamik commit'lerin incelemesi ve düzeltmeler (`ae15fc7`, `a6f8d0b`)

Dinamik analiz commit'leri (`8842e0b`, `f90281d`, `93778ab`, `ae15fc7`) bağımsız olarak incelendi. Model notu: bu projedeki tüm oturumlar pratikte **Opus 4.8** ile yürüdü (güvenlik sınıflandırıcısı Opus 5.5 yanıtlarını durdurduğu için 4.8'e düşülüyor). `ae15fc7` commit mesajındaki "Opus 5.5 gözden geçirmesi" ifadesi bu yüzden **yanlıştır**; inceleme 4.8 ile yapılmıştır.

İncelemede bulunup düzeltilen açıklar:

1. **Sessiz zayıf temizlik (`ae15fc7`).** Windows'ta Job Object ataması başarısız olursa araç **sessizce** zayıf süreç-temizliğine düşüyordu; "taranamadı ≠ güvenli" ilkesini çiğniyordu. Artık rapora açık bir **"süreç ağacı temizliği zayıf moda düştü"** uyarısı ekleniyor. Docker testine de, pickle bayt-değişiminin gerçekten uygulandığını doğrulayan bir sağlama kondu.
2. **Docker zaman aşımında konteyner sızıntısı (`a6f8d0b`).** `docker run --rm` isimsiz çalıştığından, zaman aşımında yalnızca client süreci ölüyor, konteyner arkada çalışmaya devam edebiliyordu — Docker yolunun temizlik garantisi aslında tutmuyordu. Artık benzersiz `--name` veriliyor ve zaman aşımında `docker rm -f` ile konteyner zorla kaldırılıyor.
3. **Port bulgusunda aşırı kesinlik (`a6f8d0b`).** "Yeni dinlenen port" bulgusu host modunda nedeni kanıtlanamazken kesin "arka kapı" diyordu. `new_procs` bulgusundaki dürüstlük uyarısıyla tutarlı olacak şekilde, tarama penceresinde açılan ilgisiz bir servisin de farka yol açabileceği ve izole ölçüm için Docker'ın tercih edilmesi gerektiği notu eklendi.

---

5. Diğer araçlarla karşılaştırma (projenin nicel katkısı)

Aynı etiketli 9 dosyalık korpusta (**7 zararlı, 2 zararsız**; "zararlı" örneklerin hepsi zararsız `echo` canary'si taşır) üç tarayıcı aynı ölçütlerle çalıştırıldı (`tools/compare_scanners.py`):

Araç	Precision	Recall	Yanlış alarm (FPR)
modelguard	%100	%100	%0
picklescan 1.0.5	%100	%100	%0
ModelScan 0.8.8	%100	**%43**	%0

ModelScan 7 zararlıdan 4'ünü kaçırdı (recall %43):

Kaçırdığı	ModelScan ne yaptı (kendi JSON çıktısından)	modelguard ne buldu
`.bin` uzantılı pickle	dosyayı **hiç taramadı** (`total_scanned: 0`), hata da vermedi	magic-byte ile pickle tanındı → `nt.system` MALICIOUS
kesik/yarım pickle	ayrıştırma hatası verdi, 0 bulgu	`nt.system` MALICIOUS + "akış tam çözümlenemedi" SUSPICIOUS
sıfır-CRC ZIP	**hiç taramadı** (`total_scanned: 0`), hata da vermedi	CRC doğrulamadan üyeyi okudu → `nt.system` MALICIOUS
7z paketleme	**hiç taramadı** (`total_scanned: 0`), hata da vermedi	7z açıldı → `nt.system` MALICIOUS

En önemli gözlem: ModelScan bu üç dosyada hiçbir şey taramadığı halde "0 sorun" raporladı ve uyarı vermedi. Kullanıcı bunu "temiz" diye okur. modelguard'ın "taranamadı ≠ güvenli" ilkesinin neden var olduğunu gösteren somut örnek budur. (ModelScan'in bu dosyaları *neden* atladığı, iç kodu incelenmeden kesin söylenemez; ölçülen şey, atladığı ve bunu bildirmediğidir.)

> **Dürüstlük notu (rapora yazıldı):** Korpus küçük ve sentetiktir; mutlak rakamlar genellenemez. Anlamlı olan **göreli** tablo: hangi atlatma sınıfı hangi araçtan kaçıyor. picklescan 1.0.5 güçlü bir referans (aktif bakım); modelguard'ın ayırt edici yanı atlatma dayanıklılığı **+ tek araçta çok-format + çerçeve eşlemesi + dinamik analiz**.

Ayrıntı: `reports/COMPARISON.md` . HTML: `python tools/compare_scanners.py` → `reports/comparison.html` .

---

6. Bölüm 2 — davranışsal backdoor iskeleti

Fikir: Bir modele, görüntüde belirli bir **fiziksel nesne** varken **zararsız** ama öngörülebilir bir davranış öğretmek (veri zehirleme). Sonra bu modeli Bölüm 1'den geçirip **"statik tarama temiz ama model backdoor'lu"** çelişkisini göstermek.

```
backdoor-study/
  config.py          deney ayarları + doğrulama (hedefin zararsızlığı belgeli)
  dataset.py         fotoğraf keşfi + stratified/deterministik train/val/test bölme
  poison.py          zehirli örnek seçimi + (opsiyonel) BadNets yaması
  metrics.py         CA (temiz doğruluk) + ASR (saldırı başarı oranı)
  train_classifier.py  Plan A: ResNet-18/MobileNet (8GB'ta GÜVENLİ)
  train_vlm_qlora.py   Plan B: SmolVLM/Qwen2.5-VL 4-bit QLoRA (8GB'ta riskli)
  defense_demo.py    Bölüm 1 ⇔ 2 köprüsü (statik temiz, davranış backdoor'lu)
```

İki plan, ortak iskelet

- **Plan A (sınıflandırıcı):** küçük bir görüntü sınıflandırıcıya BadNets tarzı backdoor. 8GB RTX 3070'te rahat çalışır. **Garanti yol.**
- **Plan B (VLM QLoRA):** küçük bir görsel-dil modeline 4-bit QLoRA ile. Hocanın beklentisi ama 8GB'ta OOM riski var. **Karar noktası:** ilerlemezse Plan A'ya dönülür (teori aynı).
- `config/dataset/poison/metrics` **iki planda da ortak** ve ağır bağımlılık olmadan test edilir.

İskeletin gerçek durumu (dürüst değerlendirme)

Bu kod henüz **hiç gerçek veriyle çalıştırılmadı**. Yalnızca saf mantık kısmı test edildi. İnceleme sırasında iki eksik bulunmuştu; biri düzeltildi, biri hâlâ açık:

1. **Plan A fiziksel-mod tasarım hatası — DÜZELTİLDİ (`0b8f934`).** Eski iskelette üst düzey `data/trigger/` klasörü modelin ayrı bir *sınıfı* haline geliyordu; nesneli fotoğrafların çoğu "trigger" etiketinde kalıyor, "nesne → hedef sınıf" kısayolu öğrenilemiyordu (ASR düşük çıkardı). Yeni düzen: tetikleyicili fotoğraflar her sınıfın içindeki `trigger/` alt klasöründe yer alır ve **sınıf etiketini korur**; ayrı bir "trigger" sınıfı yoktur. `discover` etiketi her zaman üst sınıf klasöründen alır, tetikleyiciyi alt klasörden işaretler; `poison_trainset` fiziksel modda yalnızca gerçek tetikleyicili örnekleri hedefe etiketler (sahte işaret atamaz, mevcut sayıyla sınırlanır); değerlendirmede fiziksel tetikleyicili test fotoğrafları temiz CA'ya girmez, yalnızca ASR'de kullanılır. Klasör düzeni artık netleştiği için fotoğraf çekimine başlanabilir (bkz. `backdoor-study/README.md`).
2. **Plan B'nin eğitim döngüsü yazılmadı — HÂLÂ AÇIK.** `train_vlm_qlora.py` veri hazırlığını ve 4-bit QLoRA kurulumunu yapıyor, sonra bilinçli olarak duruyor. Gerçek eğitim için collator + Trainer adımı eklenmeli.

`patch` modu (sayısal yama) bu hatadan hiç etkilenmiyordu.

İki metrik

- **CA (Clean Accuracy):** tetikleyici yokken doğruluk — **yüksek** kalmalı (gizlilik).
- **ASR (Attack Success Rate):** tetikleyici varken hedef davranış oranı — **yüksek** olmalı (etkinlik).
- İyi backdoor = yüksek CA + yüksek ASR. **Tam da bu yüzden statik tarama göremez:** temiz girdilerde model tamamen normaldir.

Senin yapman gereken ilk adım (kod değil)

1. Bir fiziksel tetikleyici nesne seç (ör. belirli renkli bir post-it).
2. **Fotoğraf çekmeye, yukarıdaki 1 numaralı hata düzeltilip klasör düzeni kesinleşince başla.** `backdoor-study/README.md`'deki şu anki `data/trigger/` düzeni hatalıdır.
3. Sonra: `python backdoor-study/train_classifier.py --poison-rate 0.05`.

7. Test ve kalite

104 test: 103'ü geçiyor, 1'i atlanıyor (Docker kapalıyken konteyner testi). Dağılım:

Alan	Test
pickle / keras / gguf / savedmodel / remotecode analizörleri	11 / 12 / 14 / 8 / 13
format tespiti, entegrasyon, rapor	5 / 8 / 2
dinamik (sandbox) + sysdiff	9 + 3
HTML rapor + karşılaştırma	3 + 4
Bölüm 2 çekirdek (config/dataset/poison/metrics)	12

Testler **AAA** (Arrange-Act-Assert) desenini izler. "Zararlı" test örnekleri **zararsız canary** (`echo`); gerçek zarar, ağ veya reverse shell yok; statik testler örnekleri asla yüklemez.

8. Commit haritası (zaman çizelgesi)

```
edcaf4d  Başlangıç: tarayıcı + belgeler
443fd64  remotecode yanlış alarm düzeltmesi
fd962d7  pickle: eski torch formatı
199d1f7  keras: gerçek custom kayıtlar
4040d3a  savedmodel: checkpoint op'ları LOW
9b3b035  gguf: gerçek şablonlarda yanlış alarm
42e1e66  scanner: .keras pickle taramasından da geçer
8842e0b  dynamic: host süreç ağacı temizliği
f90281d  dynamic: sysdiff (öncesi/sonrası fark)
93778ab  test: Docker backend gerçek konteynerde
598ab21  docs: HANDOFF devir belgesi
ae15fc7  dynamic: zayıf-temizlik raporlaması (bu oturum, inceleme)   ← 8.1
03927c9  report+tools: HTML rapor + araç karşılaştırması (bu oturum) ← 8.2
6c78d75  docs: HANDOFF güncelleme
58ac4c4  backdoor: Bölüm 2 iskeleti (bu oturum)                      ← 8.3
170bfd3  docs: COMPARISON ModelScan ölçülen çıktıyla düzeltildi (bu oturum)
0b8f934  backdoor: Plan A fiziksel-mod tasarım hatası düzeltildi (bu oturum)
a6f8d0b  dynamic: Docker temizlik + port bulgusu dürüstlüğü (bu oturum, inceleme)
```

9. Ne kaldı?

- **Bölüm 1 (opsiyonel):** gerçek temiz HuggingFace modelleriyle korpusu genişletip yanlış-alarm oranını daha gerçekçi ölçmek; `fickling`'i dördüncü araç olarak eklemek.
- **Bölüm 1 (inceleme):** dinamik analiz commit'lerinin bağımsız incelemesi **yapıldı** ve iki açık düzeltildi (bkz. 4.4). İnceleme Opus 4.8 ile yapıldı; ortam Opus 5.5 yanıtlarını durdurduğu için 5.5 ile ayrı bir tur pratikte mümkün olmadı.

- **Bölüm 2 (asıl kalan):** Plan A fiziksel-mod hatası ve klasör düzeni **düzeltildi** (`0b8f934`); sıradaki iş → fotoğrafları çekmek → Plan A ile eğitmek → zehirleme oranı deneyleri (%1/%5/%10) → `defense_demo.py` ile köprüyü göstermek → (mümkünse) Plan B eğitim döngüsünü yazıp denemek.
- **8.4 rapor:** deney sonuçları gelince final rapora/sunuma dönüştürmek.

---

10. Kavram sözlüğü (öğrenme yardımı)

- **pickle RCE:** pickle yüklenirken fonksiyon çağrılabilmesinden doğan uzaktan kod yürütme.
- **magic bytes:** dosyanın gerçek türünü belirleyen ilk baytlar (uzantıdan bağımsız).
- **allowlist/denylist:** "yalnızca şunlar güvenli" vs "şunlar kötü" — allowlist daha güvenli.
- **audit hook:** `sys.addaudithook` — Python'da hassas işlemleri (dosya/ağ/süreç) dinleme.
- **Job Object (Windows):** bir süreç ağacını tek birim gibi yönetip toptan sonlandırma.
- **MITRE ATLAS / OWASP ML:** AI sistemlerine yönelik saldırı/riskleri sınıflayan çerçeveler.
- **precision/recall/FPR:** doğru alarmların oranı / yakalanan tehditlerin oranı / yanlış alarm oranı.
- **veri zehirleme (data poisoning):** eğitim verisini bozarak modele gizli davranış öğretme.
- **backdoor / trigger:** tetikleyici girdi görününce ortaya çıkan gizli davranış.
- **CA / ASR:** temiz doğruluk / saldırı başarı oranı (backdoor'un gizliliği ve etkinliği).
- **QLoRA:** 4-bit nicemlenmiş modeli küçük adaptörlerle eğitme (az VRAM ile fine-tune).